

Candidate Management API

Developer Guide & API Documentation — Portfolio Sample

Document status: Sample / Fictional API

API version: v1

Base URL: `https://api.example.com/v1`

Purpose

This sample demonstrates how a technical writer can document a REST API for developers. It covers authentication, endpoints, parameters, request and response payloads, HTTP status codes, errors, cURL examples, and an OpenAPI definition excerpt.

1. API Overview

The Candidate Management API allows authorized applications to create, retrieve, update, and delete candidate records. The API uses HTTPS and JSON for request and response bodies.

Item	Value
Protocol	HTTPS
Style	REST
Data format	JSON
Version	v1
Authentication	Bearer token

2. Authentication

Send a valid access token in the **Authorization** header for protected endpoints.

`Authorization: Bearer <access-token>`

Security note: Do not place access tokens in URLs, query parameters, source-control repositories, or public screenshots.

3. Endpoint Summary

Method	Endpoint	Purpose
GET	<code>/candidates</code>	List candidates
GET	<code>/candidates/{candidateId}</code>	Get one candidate
POST	<code>/candidates</code>	Create a candidate
PATCH	<code>/candidates/{candidateId}</code>	Update candidate fields
DELETE	<code>/candidates/{candidateId}</code>	Delete a candidate

4. List Candidates

GET /candidates

Returns a paginated list of candidate records.

Query parameters

Name	Type	Required	Description
page	integer	No	Page number. Default: 1.
limit	integer	No	Number of records per page. Default: 20. Maximum: 100.
status	string	No	Filters by candidate status: active, inactive, hired.

Example request

```
curl -X GET "https://api.example.com/v1/candidates?page=1&limit=20&status=active" \
-H "Authorization: Bearer <access-token>" \
-H "Accept: application/json"
```

Example response — 200 OK

```
{
  "page": 1,
  "limit": 20,
  "total": 2,
  "data": [
    {
      "id": "can_10021",
      "firstName": "Aarav",
      "lastName": "Sharma",
      "email": "aarav@example.com",
      "status": "active"
    },
    {
      "id": "can_10022",
      "firstName": "Meera",
      "lastName": "Kapoor",
      "email": "meera@example.com",
      "status": "active"
    }
  ]
}
```

5. Get a Candidate

GET /candidates/{candidateId}

Returns the candidate identified by **candidateId**.

```
curl -X GET "https://api.example.com/v1/candidates/can_10021" \
-H "Authorization: Bearer <access-token>" \
-H "Accept: application/json"
```

Example response — 200 OK

```
{
  "id": "can_10021",
  "firstName": "Aarav",
  "lastName": "Sharma",
  "email": "aarav@example.com",
  "status": "active"
}
```

6. Create a Candidate

POST /candidates

Creates a new candidate record. The **firstName**, **lastName**, and **email** fields are required.

Request headers

```
Authorization: Bearer <access-token>
Content-Type: application/json
Accept: application/json
```

Request body

```
{
  "firstName": "Riya",
  "lastName": "Verma",
  "email": "riya.verma@example.com",
  "status": "active"
}
```

Example response — 201 Created

```
{
  "id": "can_10023",
  "firstName": "Riya",
  "lastName": "Verma",
  "email": "riya.verma@example.com",
  "status": "active",
  "createdAt": "2026-08-22T10:30:00Z"
}
```

7. Update a Candidate

PATCH /candidates/{candidateId}

Updates only the fields included in the request body.

```
curl -X PATCH "https://api.example.com/v1/candidates/can_10023" \
  -H "Authorization: Bearer <access-token>" \
  -H "Content-Type: application/json" \
  -d '{"status":"hired"}'
```

Response — 200 OK

```
{
  "id": "can_10023",
  "status": "hired"
}
```

8. Delete a Candidate

DELETE /candidates/{candidateId}

Deletes the specified candidate record.

```
curl -X DELETE "https://api.example.com/v1/candidates/can_10023" \
  -H "Authorization: Bearer <access-token>"
```

Response — 204 No Content

9. HTTP Status Codes

Code	Meaning	When used
200	OK	Successful GET, PATCH
201	Created	Resource created successfully
204	No Content	Successful DELETE
400	Bad Request	Invalid request syntax or data
401	Unauthorized	Missing or invalid token
403	Forbidden	Token is valid but lacks permission
404	Not Found	Candidate does not exist
409	Conflict	Candidate already exists
429	Too Many Requests	Rate limit exceeded
500	Internal Server Error	Unexpected server-side error

10. Error Response

Errors use a consistent JSON structure so that client applications can handle failures predictably.

```
{
  "error": {
    "code": "CANDIDATE_NOT_FOUND",
    "message": "No candidate was found for ID can_99999",
    "requestId": "req_8f42a1"
  }
}
```

11. Data Model

Field	Type	Required	Description
id	string	Response only	Unique candidate identifier.
firstName	string	Yes	Candidate's first name.
lastName	string	Yes	Candidate's last name.
email	string/email	Yes	Candidate's email address.
status	string	No	active, inactive, or hired.
createdAt	date-time	Response only	UTC creation timestamp.